SOFTWARE DESIGN DOCUMENT (SDD)

Bharat Ballot

Secure Digital Election Management Platform

Version: 1.0
Prepared By: Team RawJet
Document Type: Software Design Document

Chapter 1 — Introduction

1.1 Purpose

The purpose of this Software Design Document (SDD) is to describe the internal architecture, software design, implementation strategy, and technical organization of the Bharat Ballot platform. This document serves as a blueprint for developers, testers, maintainers, and future contributors by explaining how the system has been designed and implemented.

Unlike the Software Requirements Specification (SRS), which defines the functional and non-functional requirements of the system, this document focuses on the architectural decisions, software components, data organization, communication mechanisms, and deployment strategy used in the implementation of Bharat Ballot.

The document provides a comprehensive overview of the frontend applications, backend services, artificial intelligence module, database design, API architecture, authentication mechanisms, and cloud infrastructure that collectively enable secure digital election management.

1.2 Scope

This document covers the technical design of Bharat Ballot, including:

- Overall system architecture
- Frontend application design
- Backend architecture
- Artificial Intelligence service
- Database organization
- Authentication and authorization
- API architecture
- Deployment architecture
- Security mechanisms
- Design principles
- Future extensibility

It is intended to serve as a technical reference throughout the software lifecycle.

1.3 Intended Audience

This document is intended for:

- Software Developers
- System Architects
- Database Designers
- Testing Engineers
- Academic Evaluators
- Future Contributors
- DevOps Engineers

1.4 Design Goals

The primary design goals of Bharat Ballot are:

- Security
- Scalability
- Maintainability
- Reliability
- Performance
- Modularity
- Extensibility
- User Experience

The platform has been designed using a modular architecture so that each subsystem can evolve independently while maintaining seamless communication with other components.

1.5 Design Principles

The implementation of Bharat Ballot follows the following software engineering principles:

Modular Design

Each major functionality is implemented as an independent module.

Examples include:

- Authentication

- Elections
- Candidates
- Voters
- Constituencies
- AI Verification
- Results

Separation of Concerns

Frontend applications focus exclusively on presentation and user interaction.

Backend services manage:

- Business logic
- Authentication
- Database operations
- Validation

The AI service performs biometric verification independently.

Service-Oriented Architecture

The platform consists of independent services communicating through REST APIs.

Major services include:

- Citizen Portal
- Command Center
- Backend API
- AI Verification Service
- MongoDB Atlas
- Cloudinary

Reusability

Reusable software components are used throughout the project.

Examples include:

- Authentication middleware
- API utilities

- React components
- Database models
- Validation functions

Scalability

The architecture allows independent scaling of:

- Frontend
- Backend
- AI Service
- Database

1.6 Technology Overview

The Bharat Ballot platform has been implemented using modern web technologies.

Layer	Technology
Frontend	React + Vite
Backend	Node.js + Express
Database	MongoDB Atlas
ODM	Mongoose
Authentication	JWT + bcrypt
AI Service	Python + FastAPI
Face Recognition	InsightFace
Cloud Storage	Cloudinary
Deployment	Render
Version Control	Git + GitHub

1.7 Document Organization

This Software Design Document is organized as follows:

Chapter	Description
Chapter 1	Introduction

Chapter	Description
Chapter 2	Overall System Architecture
Chapter 3	Frontend Design
Chapter 4	Backend Design
Chapter 5	Database Design
Chapter 6	AI Service Design
Chapter 7	Authentication & Security Design
Chapter 8	API Design
Chapter 9	Deployment Design
Chapter 10	Design Decisions
Chapter 11	Future Enhancements

Chapter Summary

This chapter introduced the Software Design Document for Bharat Ballot by defining its purpose, scope, intended audience, design objectives, engineering principles, technology stack, and document organization. The following chapters describe the internal implementation of the platform, explaining how each subsystem collaborates to deliver secure, scalable, and reliable digital election management.

Chapter 2 — Overall System Architecture

2.1 Introduction

The Bharat Ballot platform follows a modular, service-oriented architecture designed to provide a secure, scalable, and maintainable digital election management system. Instead of implementing all functionalities within a single application, the platform separates responsibilities into independent software components that communicate through RESTful APIs.

This architecture enables each subsystem to perform specialized tasks while remaining loosely coupled with other components. Such an approach simplifies maintenance, improves scalability, and allows individual services to evolve independently without affecting the overall system.

The major architectural components include the Citizen Portal, Command Center, Backend API Server, Artificial Intelligence Face Verification Service, MongoDB Atlas database, and Cloudinary cloud storage.

2.2 Architectural Style

Bharat Ballot follows a **Service-Oriented Architecture (SOA)** combined with a **Client–Server Architecture**.

The system consists of multiple client applications that communicate with centralized backend services over secure HTTP-based REST APIs. The backend further integrates with external cloud services and an independent AI microservice responsible for biometric verification.

This architectural style provides the following advantages:

- Modular development
- Independent deployment of services
- High maintainability
- Improved scalability
- Better fault isolation
- Easier future enhancements

The separation of responsibilities ensures that changes made to one subsystem have minimal impact on the remaining components.

2.3 High-Level System Architecture

The Bharat Ballot platform is composed of the following major subsystems:

Citizen Portal

The Citizen Portal is a React-based frontend application that enables registered voters to:

- Authenticate using EPIC number and password.

- Complete biometric face verification.
- View active elections.
- Cast secure votes.
- View election results.
- Manage their profile.

The portal communicates exclusively with the Backend API Server through secure REST APIs.

Command Center

The Command Center serves as the administrative interface for Heads and Administrators.

Its responsibilities include:

- Election management
- Administrator management
- Candidate management
- Voter registration
- Constituency management
- Dashboard monitoring
- Result publication

The Command Center uses the same backend API while exposing functionality based on the authenticated user's role.

Backend API Server

The Backend API Server is the core processing unit of Bharat Ballot.

Developed using **Node.js** and **Express.js**, it is responsible for:

- Business logic execution
- Authentication
- Authorization
- Database operations
- Election lifecycle management
- Communication with AI services
- Communication with Cloudinary
- REST API management

All requests originating from the frontend applications are processed by the backend before interacting with the database or external services.

AI Face Verification Service

A dedicated Python-based AI microservice performs biometric identity verification.

The service uses:

- FastAPI
- InsightFace
- OpenCV
- ONNX Runtime

Its responsibilities include:

- Face enrollment
- Facial embedding generation
- Live face verification
- Cosine similarity comparison
- Verification response generation

Separating the AI service from the backend improves maintainability and enables future upgrades to the recognition model without affecting the rest of the platform.

MongoDB Atlas

MongoDB Atlas serves as the primary persistent storage system.

It stores:

- User accounts
- Elections
- Candidates
- Constituencies
- Voters
- Participation records
- Administrative data
- Election results

Mongoose ODM is used for schema management and database communication.

Cloudinary

Cloudinary is used for cloud-based image management.

Stored assets include:

- Voter photographs
- Candidate photographs
- Processed profile images

Cloudinary provides optimized image delivery while reducing storage requirements on the application server.

2.4 System Interaction

The interaction between major components follows the sequence below:

1. A user accesses either the Citizen Portal or Command Center.
2. The frontend sends requests to the Backend API Server.
3. The backend authenticates the user and validates authorization.
4. Business logic is executed based on the requested operation.
5. Database operations are performed using MongoDB Atlas.
6. If biometric verification is required, the backend communicates with the AI Face Verification Service.
7. If image storage or retrieval is required, the backend interacts with Cloudinary.
8. The backend returns the processed response to the frontend.
9. The frontend displays the result to the user.

This layered interaction minimizes direct dependencies between clients and external services.

2.5 Request Processing Workflow

Every user request follows a standardized processing pipeline:

Step 1: User Action
The user initiates an action through the web interface.

↓

Step 2: Frontend Validation
Basic validation is performed within the React application.

↓

Step 3: API Request
The frontend sends an authenticated REST request.

↓

Step 4: Authentication & Authorization
JWT tokens are validated, and user permissions are checked.

↓

Step 5: Business Logic Execution
The backend executes the requested operation.

↓

Step 6: External Service Communication (if required)

- MongoDB Atlas
- AI Service
- Cloudinary

↓

Step 7: Response Generation

↓

Step 8: Frontend Rendering

2.6 Layered Architecture

The Bharat Ballot platform follows a layered software architecture.

Presentation Layer

Implemented using React and Vite.

Responsibilities:

- User Interface
- Form validation
- Navigation
- Dashboard rendering
- API communication

Application Layer

Implemented using Node.js and Express.

Responsibilities:

- Business logic
- Authentication

- Authorization
- Request handling
- Validation
- Route management

Service Layer

Responsible for:

- AI Face Verification
- Image Management
- Cloud integrations

Data Layer

Implemented using MongoDB Atlas.

Responsibilities:

- Persistent storage
- Data retrieval
- Transaction management
- Relationship management

2.7 Architectural Advantages

The adopted architecture provides several benefits.

Security

- JWT Authentication
- Role-Based Access Control
- AI Face Verification
- Secure REST APIs

Scalability

Independent scaling of:

- Frontend applications
- Backend server

- AI microservice
- Database

Maintainability

Each module has a clearly defined responsibility, simplifying updates and debugging.

Extensibility

New modules such as notification systems, analytics, or mobile applications can be integrated without major architectural changes.

Reliability

The separation of services reduces the impact of component failures and improves overall system stability.

2.8 Architecture Diagram

Figure 2.1 – Overall System Architecture

(Insert the complete architecture diagram created in Draw.io here.)

The architecture diagram illustrates the interaction between the Citizen Portal, Command Center, Backend API Server, AI Face Verification Service, MongoDB Atlas, and Cloudinary. It provides a visual representation of the system's service-oriented architecture and the communication pathways between each component.

Chapter Summary

This chapter presented the overall architecture of the Bharat Ballot platform, describing its service-oriented design, major software components, layered architecture, request processing workflow, and interactions with external services. The adopted architecture ensures modularity, scalability, maintainability, and security while supporting the complete digital election lifecycle. It establishes the technical foundation for the detailed subsystem designs discussed in the subsequent chapters.

Chapter 3 — Frontend Design

3.1 Introduction

The frontend of Bharat Ballot provides the user-facing interfaces through which different categories of users interact with the election management system.

The frontend architecture is implemented using **React and Vite** and follows a component-based design approach. The interface is divided according to user roles and responsibilities, while common components and services are reused wherever possible.

The frontend communicates with the Backend API Server through REST APIs and does not directly access the MongoDB database or AI service.

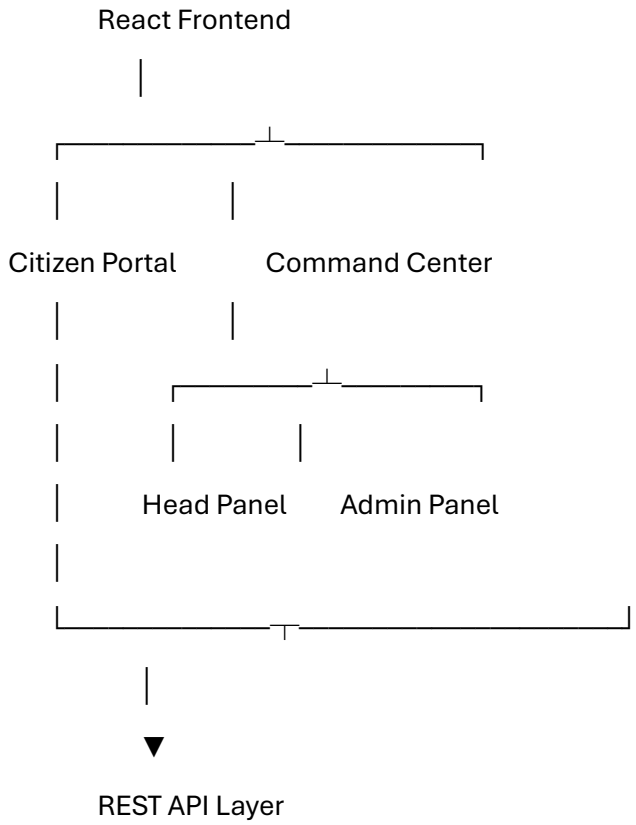
The major frontend interfaces are:

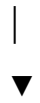
- **Citizen Portal**
- **Command Center**
    - Head Panel
    - Administrator Panel

3.2 Frontend Architecture

The frontend follows a modular component-based architecture.

At a high level, the structure can be represented as:





Each portal provides functionality specific to its intended user role while sharing common backend services.

3.3 Citizen Portal

The Citizen Portal is the voter-facing application of Bharat Ballot.

It is designed to provide voters with a simple and secure interface for participating in elections.

The major functions of the Citizen Portal include:

- Voter authentication
- Election selection
- Face verification
- Candidate viewing
- Vote casting
- Voting-status management
- Result viewing
- Profile management

The portal communicates with the Backend API Server for authentication, election information, candidate information, voting operations, and result retrieval.

3.4 Command Center

The Command Center is the administrative frontend of Bharat Ballot.

It provides centralized interfaces for managing elections and monitoring election-related activities.

The Command Center is divided into two role-specific interfaces:

3.4.1 Head Panel

The Head Panel provides system-level election management capabilities.

Major functions include:

- Dashboard overview
- Election creation and management
- Administrator management

- Election result viewing
- Election configuration
- Profile management
- Election selection

The Head Authority can manage multiple elections through the global election-selection mechanism.

3.4.2 Administrator Panel

The Administrator Panel provides election and constituency-level management capabilities.

Major functions include:

- Dashboard monitoring
- Voter management
- Candidate management
- Constituency-specific operations
- Result viewing
- Administrator profile management

Administrators are associated with a specific election and constituency, which restricts their operational scope.

3.5 React Component Architecture

The frontend uses reusable React components to reduce duplication and improve maintainability.

The Command Center follows a modular structure similar to:

```
src/
│
├── components/
│   ├── auth/
│   │   └── Guard.jsx
│   │
│   ├── common/
│   │   ├── Button.jsx
│   │   ├── Card.jsx
```

```
│   │   ├── Confirm.jsx
│   │   ├── Field.jsx
│   │   ├── Modal.jsx
│   │   ├── Page.jsx
│   │   ├── Select.jsx
│   │   ├── Status.jsx
│   │   └── Toast.jsx
│   │
│   ├── layout/
│   │   └── PortalShell.jsx
│   │
│   └── pages/
│       ├── AdminOverview.jsx
│       ├── Admins.jsx
│       ├── Directory.jsx
│       ├── Elections.jsx
│       ├── HeadOverview.jsx
│       ├── Profile.jsx
│       └── SettingsPage.jsx
│
├── context/
│   └── ElectionContext.jsx
│
├── services/
│   └── api.js
│
└── App.jsx
```

This structure separates reusable interface elements, authentication components, layouts, pages, application state, and API communication.

3.6 Common UI Components

Common components are implemented separately so that the same interface patterns can be reused across multiple pages.

Button

The Button component provides a reusable button interface with consistent styling and behavior.

Card

The Card component provides a standardized container for displaying grouped information.

Modal

The Modal component provides a reusable overlay interface for forms, information, and administrative operations.

Confirm

The Confirm component provides confirmation dialogs for potentially destructive operations such as deletion or reset operations.

Field

The Field component provides standardized form input presentation.

Select

The Select component provides reusable selection controls.

Status

The Status component displays election or entity status information consistently.

Toast

The Toast component provides temporary feedback messages after operations such as successful creation, deletion, updates, or API failures.

3.7 Layout Architecture

The Command Center uses a common layout structure through the PortalShell component.

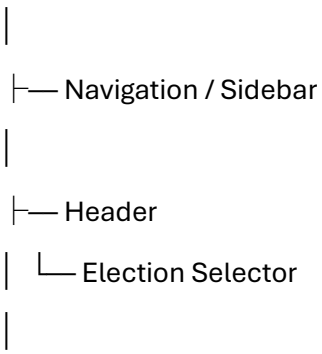
The layout provides shared elements such as:

- Navigation
- Sidebar
- Header
- Election selector
- Main content area
- Footer

Individual pages are rendered inside the shared portal shell.

The general structure is:

PortalShell



This prevents each page from implementing its own navigation and layout structure.

3.8 Routing Design

React Router is used to implement client-side routing.

Routes are protected according to the user's role.

The routing structure includes separate paths for Head and Administrator functionality.

Examples include:

/head

/head/dashboard

/head/election

/head/viewAdmins

/head/results

/head/settings

/admin

/admin/dashboard

/admin/voters

/admin/candidate/view

/admin/results

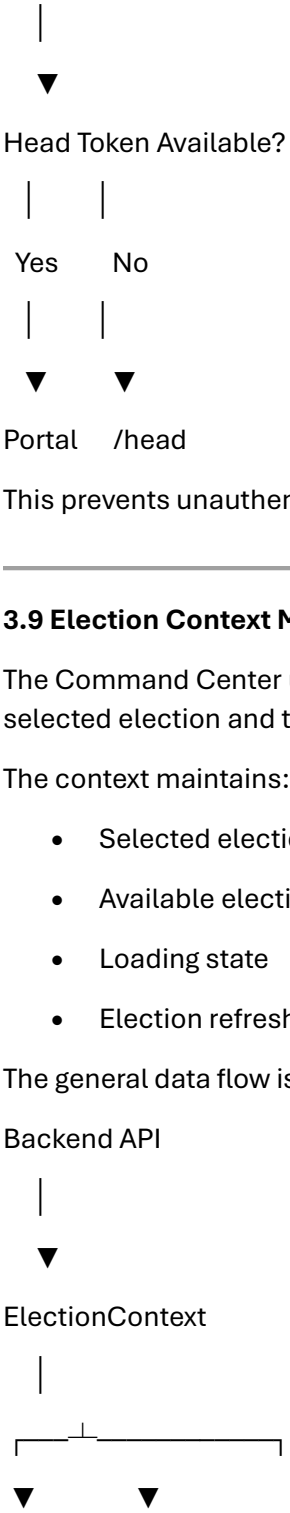
/admin/profile

Protected routes are wrapped using the Guard component.

The Guard checks whether the appropriate authentication token exists in local storage.

For example:

Head Route



This prevents unauthenticated users from directly accessing protected portal pages.

3.9 Election Context Management

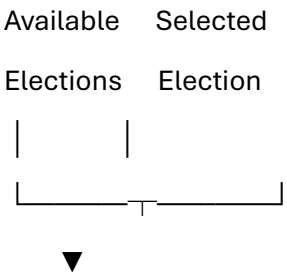
The Command Center uses a React Context named ElectionContext to manage the currently selected election and the elections available to the Head Authority.

The context maintains:

- Selected election
- Available elections
- Loading state
- Election refresh operation

The general data flow is:

Backend API



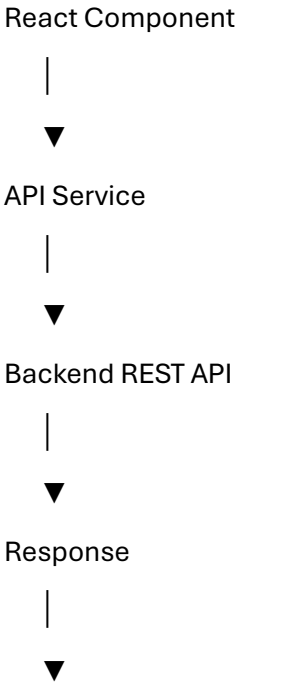
This avoids passing election information through multiple levels of React components and allows different pages to access the currently selected election when required.

3.10 API Service Integration

Frontend API communication is centralized through the API service.

Instead of implementing HTTP communication separately in every component, pages use the shared API utility to communicate with the Backend API Server.

The general flow is:



This approach provides a consistent mechanism for:

- API requests
- Authentication information
- Response handling
- Error handling

- Backend communication

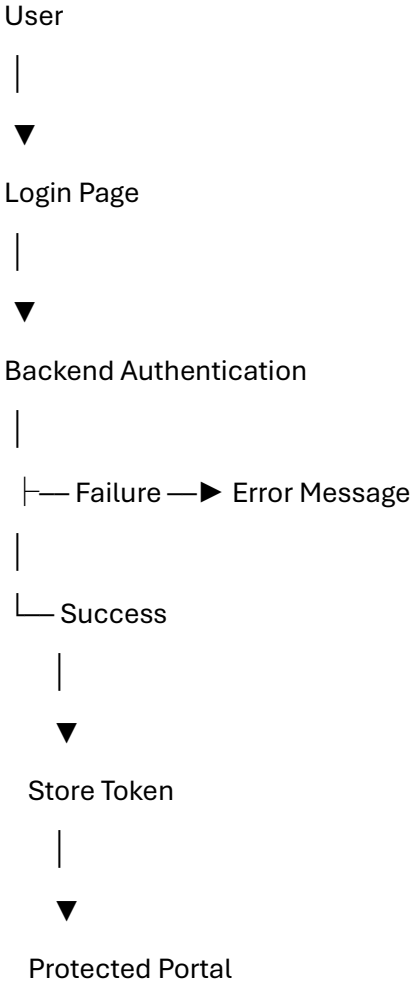
3.11 Frontend Authentication Design

Authentication is implemented separately for the Head and Administrator roles.

After successful authentication, the corresponding token is stored locally.

The frontend then uses the appropriate token to access protected APIs and protected routes.

The authentication flow can be represented as:



The Guard component provides an additional frontend-level protection mechanism for protected routes.

Backend authorization remains responsible for enforcing actual access permissions.

3.12 Frontend Design Principles

The frontend implementation follows the following principles:

Component Reusability

Common UI functionality is implemented once and reused across pages.

Separation of Responsibilities

Authentication, layout, common UI elements, pages, state management, and API communication are separated into appropriate modules.

Role-Based Interface

Users receive interfaces appropriate to their assigned role.

Consistent User Experience

Reusable components provide consistent styling and interaction patterns throughout the Command Center.

Maintainability

The modular React structure allows individual pages and components to be modified without requiring changes to unrelated parts of the application.

3.13 Frontend Data Flow

The overall frontend data flow is:

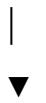
User Interaction

React Component

API Service

Backend API

Database / External Service



API Response

▼

React State / Context

|

▼

UI Update

For election-related operations, ElectionContext can maintain the selected election and make the information available to relevant components.

Chapter Summary

This chapter described the frontend design of Bharat Ballot. The frontend follows a modular React-based architecture consisting of the Citizen Portal and Command Center, with separate Head and Administrator interfaces. Reusable components, protected routes, centralized API communication, and React Context are used to improve maintainability, consistency, and separation of concerns.

The next chapter will describe the **Backend Design**, including the Node.js/Express architecture, route organization, middleware, authentication, business logic, and communication with MongoDB, Cloudinary, and the AI service.

Chapter 4 — Backend Design

4.1 Introduction

The Backend API Server forms the central processing layer of the Bharat Ballot platform. It is responsible for handling requests from the Citizen Portal and Command Center, executing business logic, validating data, enforcing authentication and authorization, and communicating with the database and external services.

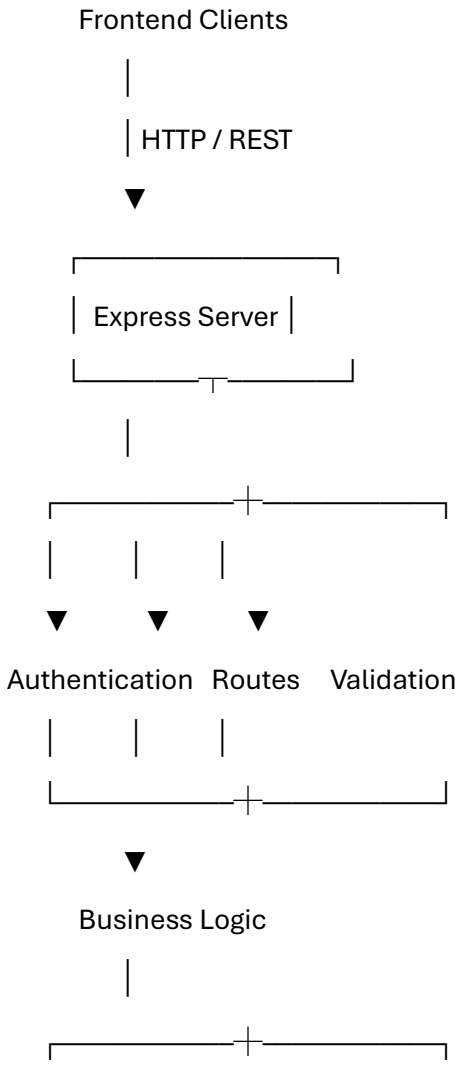
The backend is implemented using **Node.js and Express.js**. MongoDB Atlas is used as the primary database, with **Mongoose** providing schema definitions and database interaction.

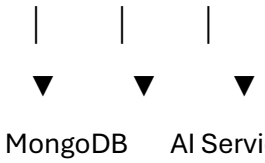
The backend follows a modular REST API design in which different functional areas are exposed through dedicated API routes.

4.2 Backend Architecture

The backend follows a layered architecture consisting of request handling, authentication and authorization, business logic, database interaction, and external service integration.

The simplified architecture is:





The backend acts as the intermediary between frontend applications and the underlying data and external services.

4.3 Backend Responsibilities

The Backend API Server performs the following major responsibilities:

- User authentication
- Role-based authorization
- Election management
- Constituency management
- Administrator management
- Candidate management
- Voter management
- Voting operations
- Participation tracking
- Result processing
- Election lifecycle management
- Face-verification communication
- Image management
- Input validation
- Database operations
- API response generation
- Error handling

Centralizing these responsibilities prevents frontend clients from directly accessing sensitive application resources.

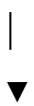
4.4 Express.js Application Structure

Express.js is used as the HTTP server framework.

Incoming requests are processed through the Express application and routed according to the requested endpoint.

The general request-processing structure is:

Client Request



Express Application

Middleware

Route Handler

Business Logic

Database / External Service

Response

This structure allows common processing operations to be applied consistently across multiple endpoints.

4.5 API Route Organization

The backend exposes REST-style endpoints grouped according to system functionality.

Major API groups include:

Authentication APIs

Responsible for authenticating:

- Head Authority
- Administrators
- Voters

Authentication endpoints validate credentials and generate the appropriate authentication token.

Head APIs

Head-level endpoints provide system-wide election management capabilities.

These operations include:

- Election creation
- Election modification
- Election deletion
- Election activation
- Election reset
- Administrator management
- Result publication
- Result viewing
- Dashboard information

Administrator APIs

Administrator endpoints provide constituency-level management functionality.

These include:

- Administrator profile access
- Voter registration
- Voter modification
- Voter deletion
- Candidate registration
- Candidate modification
- Candidate deletion
- Constituency-specific information

- Result viewing

Voter APIs

Voter endpoints support operations required during voter participation.

These include:

- Voter authentication
- Election information retrieval
- Candidate retrieval
- Face verification
- Vote submission
- Participation status
- Result access

4.6 Authentication and Authorization Layer

The backend uses token-based authentication to identify authenticated users.

After successful authentication, the backend generates an authentication token which is subsequently supplied with protected requests.

The backend verifies the token before allowing access to protected resources.

Authorization is then applied according to the user's role.

The major roles are:

Head

│

└── System / Election Management

Admin

│

└── Election + Constituency Management

Voter

│

└── Voting Operations

Authentication confirms **who the user is**, while authorization determines **what that user is allowed to access**.

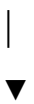
4.7 Election Management Design

The Election module manages the complete lifecycle of an election.

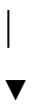
An election can progress through different operational states such as:

Draft

Active



Completed



Results Published

The backend controls transitions between these states and validates whether requested operations are permitted for the current election state.

Election records act as the primary reference for election-specific data.

Entities such as administrators, candidates, constituencies, and participation records contain an association with the relevant election.

4.8 Constituency Management

Bharat Ballot separates **master constituency information** from election-specific constituency records.

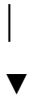
The MasterConstituency collection contains reusable constituency definitions.

When an election is created, the required constituency records can be generated for that specific election.

The relationship can be represented as:

MasterConstituency

Election



Election-Specific Constituency

This design allows the same master constituency information to be reused across different elections while maintaining separate election-specific records.

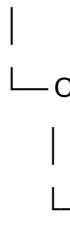
4.9 Administrator Management

Administrators are assigned to specific elections and constituencies.

The backend maintains these relationships so that an administrator cannot manage unrelated election data.

The logical association is:

Election



Constituency

Administrator

Administrator operations are therefore constrained by their assigned election and constituency.

This provides an additional layer of authorization beyond simply checking whether the user is an administrator.

4.10 Candidate Management

The Candidate module manages candidates participating in a particular election and constituency.

The backend supports:

- Candidate registration
- Candidate modification
- Candidate deletion
- Candidate information retrieval
- Candidate image management
- Party-symbol/image management
- Criminal declaration information

Candidate records are associated with their relevant election and constituency, ensuring that candidates belonging to one election do not appear in unrelated elections.

4.11 Voter Management

The Voter module manages registered voter information.

The backend supports:

- Voter registration
- Voter information updates
- Voter deletion where permitted
- Voter authentication
- Voter election association
- Voter photograph processing
- Face enrollment
- Voting-status management

Voters are treated as persistent users of the platform, while their participation in individual elections is tracked separately.

This allows the same voter account to participate in multiple elections without creating a new permanent account for every election.

4.12 Participation Management

The Participation entity is used to track a voter's participation in a particular election.

The relationship can be represented as:

Voter

```
 │
 └── Participation
      │
      └── Election
```

This separates the voter's permanent identity from election-specific participation data.

Participation records can therefore be used to determine whether a voter has already participated in a particular election.

4.13 Voting Processing

The voting operation is processed through the backend rather than allowing the frontend to directly modify election data.

A simplified voting flow is:

Voter

Before accepting a vote, the backend can validate the authenticated voter, election state, participation status, and other required conditions.

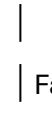
4.14 Face Verification Integration

The Node.js backend communicates with the independent Python AI service when biometric verification is required.

The backend acts as the intermediary between the frontend and AI service.

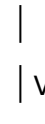
The communication flow is:

Voter Portal



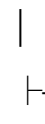
Face Image

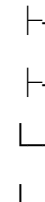
Node.js Backend



Verification Request

Python AI Service





Face Detection

Embedding Generation

Similarity Comparison

Verification Result

Node.js Backend

Voter Portal

The AI service returns a verification result to the backend, which then determines whether the voting process can proceed.

4.15 Image Management

Image-related operations are handled through the backend and integrated with cloud storage where required.

Images can include:

- Voter photographs
- Candidate photographs
- Profile images
- Party symbols

The backend processes uploaded images and communicates with Cloudinary for cloud-based storage and retrieval.

This prevents frontend applications from directly managing cloud credentials or storage operations.

4.16 Database Communication

Mongoose is used as the Object Data Modeling layer between the Node.js application and MongoDB Atlas.

Mongoose provides:

- Schema definitions
- Data validation
- Model abstraction
- References between collections
- Query operations
- Index definitions
- Database lifecycle operations

The general flow is:

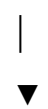
API Request

Express Route

Mongoose Model

MongoDB Atlas

Database Response



API Response

4.17 Election Reset Design

The backend provides an election reset operation to remove election-specific data while retaining permanent voter information and master constituency information.

The reset operation can remove records associated with the selected election, including:

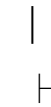
- Participation records
- Candidates
- Administrators
- Election-specific constituencies
- Election record

The operation is performed using a database transaction to maintain consistency.

The conceptual process is:

Reset Election

Start Transaction



├— Delete Participation

├— Delete Candidates

├— Delete Administrators

├— Delete Constituencies

└── Delete Election

│

▼

Commit Transaction

If an error occurs during the operation, the transaction can be aborted to prevent an incomplete reset.

4.18 Error Handling

The backend returns appropriate HTTP responses when operations fail.

Common error conditions include:

- Invalid authentication
- Unauthorized access
- Missing resources
- Invalid input
- Duplicate records
- Database failures
- External-service failures
- Invalid election state

Errors are returned to the frontend in a structured response so that the user interface can display appropriate feedback.

4.19 Backend Design Principles

The backend follows several important design principles:

Separation of Concerns

Authentication, routing, business logic, database operations, and external service communication are logically separated.

Server-Side Validation

Important validation is performed on the backend rather than relying solely on frontend validation.

Role-Based Access

Operations are restricted according to the authenticated user's role and assigned election/constituency.

Data Isolation

Election-specific records are associated with their corresponding election.

Transactional Operations

Critical multi-step database operations, such as election reset, use transactions to preserve consistency.

External Service Isolation

AI processing and cloud storage are accessed through controlled backend integrations rather than directly from the frontend.

Chapter Summary

This chapter described the backend architecture of Bharat Ballot and its role as the central processing layer of the platform. The Node.js and Express-based backend manages authentication, authorization, election lifecycle operations, constituency management, voter and candidate management, voting operations, database communication, image management, and integration with the independent AI face-verification service.

Chapter 5 — Database Design

5.1 Introduction

The database layer of Bharat Ballot is responsible for storing and managing the persistent information required by the election management platform.

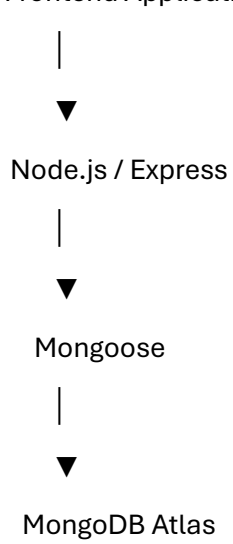
Bharat Ballot uses **MongoDB Atlas** as its primary database and **Mongoose** as the Object Data Modeling (ODM) layer. The database design follows a document-oriented approach while maintaining logical relationships between election, constituency, administrator, candidate, voter, and participation data.

The database is designed to support multiple elections while maintaining appropriate separation between election-specific records and permanent voter information.

5.2 Database Architecture

The application communicates with MongoDB Atlas through the backend server.

The frontend applications do not communicate directly with the database.



This architecture ensures that database credentials, validation rules, and database operations remain controlled by the backend.

5.3 Database Design Approach

The database design follows the following principles:

- Separation of permanent and election-specific data
- Referential relationships through MongoDB ObjectIds
- Schema validation through Mongoose
- Election-based data isolation

- Reusable master data
- Indexed fields for frequently used queries
- Transaction support for critical multi-document operations

A central design decision is the separation between **Master Constituency** data and **election-specific Constituency** data.

5.4 Major Collections

The major collections used by Bharat Ballot include:

Collection	Purpose
Election	Stores election information and lifecycle state
MasterConstituency	Stores reusable constituency definitions
Constituency	Stores election-specific constituency records
Admin	Stores election administrators
Candidate	Stores candidates participating in elections
Voter	Stores permanent voter accounts and voter information
Participation	Tracks voter participation in individual elections

These collections collectively represent the core election-management data model.

5.5 Election Collection

The Election collection represents an individual election conducted through the platform.

An election acts as the primary reference for election-specific data.

The collection stores information such as:

- Election title
- Election type
- Geographic information
- Start date and time
- End date and time
- Election status
- Result publication state
- Election configuration information

The election identifier is referenced by other election-specific collections.

The logical relationship is:

Election

│

├— Constituencies

├— Administrators

├— Candidates

└— Participation Records

5.6 MasterConstituency Collection

The MasterConstituency collection contains reusable constituency definitions.

It acts as the master reference from which election-specific constituency records can be created.

The purpose of this collection is to avoid duplicating the complete definition of a constituency every time a new election is created.

The relationship can be represented as:

MasterConstituency

│

│ referenced by

▼

Constituency

│

│ associated with

▼

Election

This design also allows constituency information to remain consistent across different elections.

5.7 Constituency Collection

The Constituency collection represents a constituency within a particular election.

Unlike MasterConstituency, these records are election-specific.

Important fields include:

- name
- state
- district
- city
- masterConstituencyId
- electionId
- election
- constituencyNumber
- constituencyName
- active

The electionId associates the constituency with its election.

The schema also defines a unique compound index:

{ electionId: 1, name: 1 }

This ensures that the same constituency name cannot be duplicated within the same election.

A separate index is maintained for:

{ electionId: 1, constituencyNumber: 1 }

allowing efficient election-specific constituency-number queries.

5.8 Administrator Collection

The Admin collection stores administrator accounts.

An administrator is associated with a particular election and constituency.

The logical relationship is:

Election

```
  │
  └── Constituency
        │
        └── Admin
```

This relationship is important for enforcing constituency-level administrative access.

Administrators can therefore operate within the scope assigned to them rather than receiving unrestricted access to every election or constituency.

5.9 Candidate Collection

The Candidate collection stores information about candidates participating in an election.

Candidate information may include:

- Candidate name
- Candidate identifier
- Election association
- Constituency association
- Candidate photograph
- Party symbol/image
- Criminal declaration information

Candidate records are associated with the relevant election so that candidates from one election remain isolated from candidates belonging to another election.

5.10 Voter Collection

The Voter collection stores the permanent voter account and identity information.

The voter model contains information required for:

- Authentication
- Identification
- Personal information
- Contact information
- Address information
- Constituency associations
- Photograph
- Face-verification information
- Account status

Voters are intentionally treated differently from election-specific administrative data.

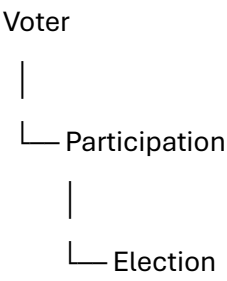
A voter account can remain available for future elections rather than being deleted when an individual election is reset.

5.11 Participation Collection

The Participation collection stores election-specific voter participation information.

It separates the permanent voter identity from their participation in a particular election.

The logical relationship is:

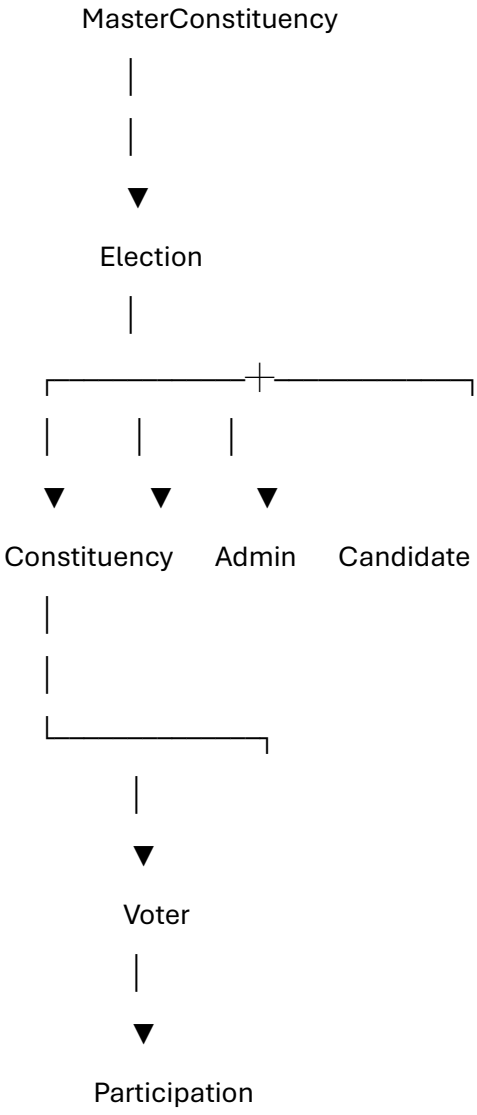


This allows the system to determine whether a particular voter has already participated in a particular election.

It also supports the design principle that voter accounts remain persistent while participation records are specific to individual elections.

5.12 Entity Relationships

The major database relationships can be represented as:



The actual implementation uses MongoDB ObjectId references where relationships between documents are required.

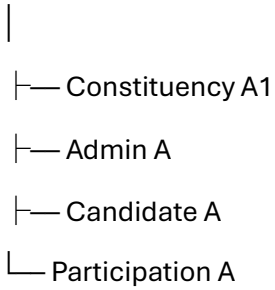
5.13 Election-Specific Data Isolation

Election isolation is an important part of the database design.

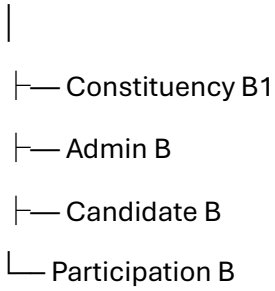
Records belonging to an election contain an electionId reference where appropriate.

For example:

Election A



Election B



This allows the backend to retrieve and manipulate data belonging to a specific election without mixing records from other elections.

5.14 Permanent and Temporary Election Data

The database distinguishes between permanent user information and election-specific information.

Persistent Data

Examples include:

- Voter account

- Voter identity information
- Face profile/embedding
- Master constituency information

Election-Specific Data

Examples include:

- Election record
- Election-specific constituencies
- Administrators
- Candidates
- Participation records

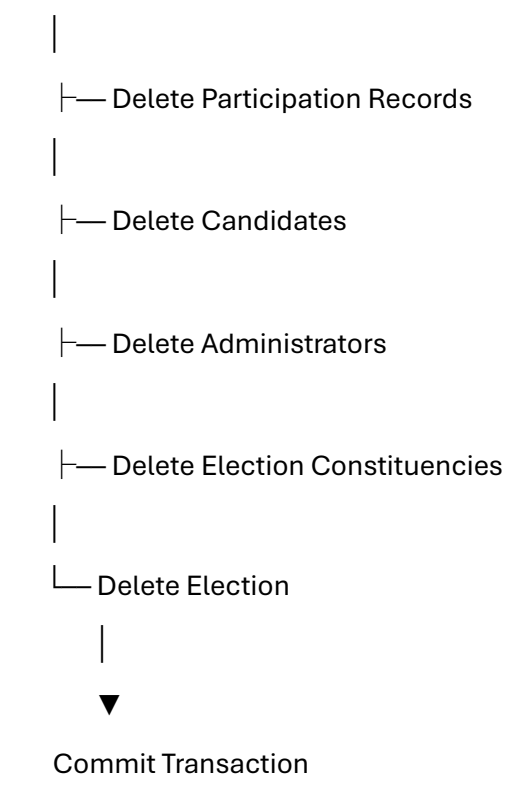
This distinction is particularly important during the election reset operation.

5.15 Election Reset and Database Transactions

Bharat Ballot uses a database transaction for critical election-reset operations.

During an election reset, election-specific records can be removed together as one logical operation.

The process is:



If an operation fails, the transaction can be aborted.

This reduces the possibility of leaving the database in an inconsistent intermediate state.

5.16 Indexing Strategy

Indexes are used to improve query performance and enforce uniqueness constraints.

The Constituency collection includes an index on:

electionId + name

with a unique constraint.

It also includes an election-specific constituency-number index:

electionId + constituencyNumber

with sparse indexing.

These indexes support common election-specific constituency queries while preventing unwanted duplicate constituency names within the same election.

5.17 Mongoose Integration

Mongoose provides the interface between the Express backend and MongoDB Atlas.

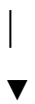
Schemas define:

- Field types
- Required fields
- Default values
- References
- Validation rules
- Indexes
- Timestamps

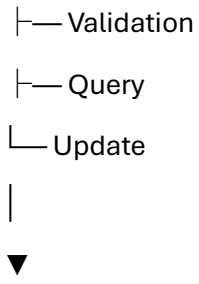
Mongoose models are then used by backend routes and services to perform database operations.

The interaction can be represented as:

Express Route



Mongoose Model



5.18 Data Integrity

Data integrity is maintained through multiple mechanisms:

Schema Validation

Required fields and field types are defined through Mongoose schemas.

References

MongoDB ObjectIds are used to associate related records.

Unique Constraints

Compound indexes prevent specific duplicate records.

Election Association

Election-specific entities are associated with their corresponding election.

Transactions

Critical multi-step operations use transactions where required.

5.19 Database Security

The database is accessed only through the backend application.

Frontend clients are not provided with direct MongoDB credentials.

MongoDB Atlas provides the managed database infrastructure while the backend controls access to database operations.

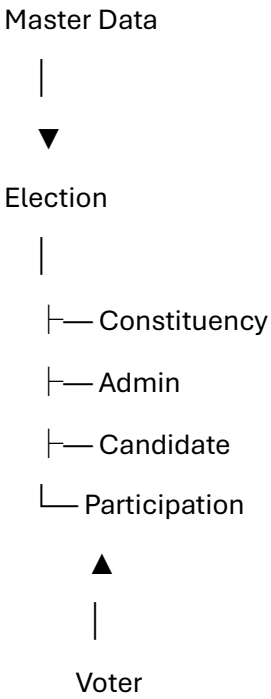
This architecture helps protect:

- User information
- Election information
- Candidate information
- Participation records
- Authentication-related data

5.20 Database Design Summary

The Bharat Ballot database is designed around the central Election entity while maintaining reusable master constituency data and persistent voter accounts.

The primary design relationship is:



This structure provides election-level isolation, supports multiple elections, allows voter accounts to remain persistent, and provides a foundation for secure election management.

Chapter Summary

This chapter described the database architecture and data organization of Bharat Ballot. MongoDB Atlas is used as the primary persistence layer, with Mongoose providing schema management and database interaction. The design separates reusable master constituency data, permanent voter information, and election-specific records.

The use of election references, ObjectId relationships, indexes, schema validation, and transactional operations provides the foundation for maintaining data consistency and supporting multiple elections within the same platform.